Introduction

Angular is a modern front-end framework developed by Google for building dynamic and scalable web applications. It follows a component-based architecture and provides powerful features such as routing, dependency injection, reactive programming, forms, HTTP communication, and state management.

The Student Course Portal developed throughout these exercises demonstrates real-world Angular concepts including:

- Components
- Data Binding
- Directives
- Pipes
- Forms
- Services
- Routing
- HTTP Communication
- RxJS
- NgRx
- Unit Testing

By completing these exercises, students gain practical knowledge required for enterprise Angular development.

Software Requirements

Hardware Requirements

- Intel i3 Processor or above
- 8 GB RAM
- 20 GB Free Disk Space

Software Requirements

Operating System

- Windows 10 / Windows 11

IDE

- Visual Studio Code

Runtime

- Node.js Latest LTS

Framework

- Angular CLI 20

Language

- TypeScript

Backend

- JSON Server

Package Manager

- npm

Browser

- Google Chrome

Testing Framework

- Jasmine

Test Runner

- Karma

State Management

- NgRx

Version Control

- Git

Installing Required Software

Install Node.js

Download and install the latest LTS version from the official Node.js website.

Verify installation

node -v

npm -v

Install Angular CLI

npm install -g @angular/cli

Verify

ng version

Install VS Code Extensions

Install the following extensions:

- Angular Language Service
- TypeScript Hero
- Prettier
- ESLint
- GitLens

Create Angular Project

ng new student-course-portal

Choose

CSS

Routing : Yes

Move inside project

cd student-course-portal

Run

ng serve

Open browser

http://localhost:4200

Angular Project Structure

```
student-course-portal
│
├── src
│   ├── app
│   │   ├── components
│   │   ├── pages
│   │   ├── services
│   │   ├── directives
│   │   ├── guards
│   │   ├── interceptors
│   │   ├── layouts
│   │   ├── models
│   │   ├── pipes
│   │   ├── store
│   │   ├── app.config.ts
│   │   └── app.routes.ts
│
├── assets
├── package.json
└── angular.json
```

Exercise 1

Angular Components and Data Binding

Objective

The objective of this exercise is to understand Angular components, data binding techniques, interpolation, property binding, event binding, two-way binding, lifecycle hooks, and component interaction.

Learning Outcomes

After completing this exercise, students will be able to:

- Create Angular components.
- Understand Angular project structure.
- Use interpolation.
- Implement property binding.
- Implement event binding.
- Use two-way binding.
- Work with lifecycle hooks.
- Create reusable UI components.

Components to Create

Generate the following components.

ng generate component components/header

ng generate component pages/home

ng generate component pages/course-list

ng generate component pages/student-profile

ng generate component components/course-card

Folder Structure After Exercise 1

```
app
 ├── components
 │   ├── header
 │   └── course-card
 │
 ├── pages
 │   ├── home
 │   ├── course-list
 │   └── student-profile
 │
 ├── app.component.ts
 ├── app.component.html
 └── app.routes.ts
```

Understanding Angular Components

Angular applications are built using components.

Each component consists of

- TypeScript class
- HTML template
- CSS stylesheet

Example

HomeComponent

home.component.ts

home.component.html

home.component.css

Each component controls a small portion of the user interface, making Angular applications modular, reusable, and maintainable.

Types of Data Binding

Angular provides four major types of data binding.

Interpolation

Displays data from TypeScript to HTML.

Example

<h1>{{ portalName }}</h1>

Property Binding

Binds HTML properties.

Example

<img [src]="logo">

Event Binding

Captures user actions.

Example

<button (click)="enroll()">

Enroll

</button>

Two-Way Binding

Synchronizes data between HTML and TypeScript.

Example

<input [(ngModel)]="studentName">

Exercise 1 – Angular Components and Data Binding (Implementation)

Step 1: Create Angular Project

Create a new Angular project using the Angular CLI.

ng new student-course-portal --routing --style=css

Move into the project directory.

cd student-course-portal

Run the application.

ng serve

Open the application in a browser.

http://localhost:4200

Step 2: Generate Components

Generate all required components.

ng generate component components/header

ng generate component pages/home

ng generate component pages/course-list

ng generate component pages/student-profile

ng generate component components/course-card

Step 3: Configure Routing

Create routes for all pages.

app.routes.ts

```
import { Routes } from '@angular/router';


import { HomeComponent } from './pages/home/home.component';
import { CourseListComponent } from './pages/course-list/course-list.component';
import { StudentProfileComponent } from './pages/student-profile/student-profile.component';
```

```
export const routes: Routes = [
{
path:'',
component:HomeComponent
},

{
path:'courses',
component:CourseListComponent
}
{
path:'profile',
component:StudentProfileComponent
}
];
```

Step 4: Root Component

app.component.html

```
<app-header></app-header>
<router-outlet></router-outlet>
```

This serves as the root layout of the application. The Header component is displayed on every page, while the Router Outlet dynamically loads the selected page.

Step 5: Header Component

header.component.ts

```
import { Component } from '@angular/core';
```

```
@Component({
selector:'app-header',
standalone:true,
templateUrl:'./header.component.html',
styleUrls:['./header.component.css']
})
export class HeaderComponent{
title='Student Course Portal';
}
```

---

header.component.html

```
<header>
<h1>
{{title}}
</h1>
<nav>
<a routerLink="/">Home</a>
<a routerLink="/courses">Courses</a>
<a routerLink="/profile">Profile</a>
</nav>
</header>
```

---

header.component.css

```
header{
background:#1976d2;
color:white;
padding:20px;
```

```
}
nav{
margin-top:15px;
}
nav a{
color:white;
margin-right:20px;
text-decoration:none;
font-weight:bold;
}
nav a:hover{
text-decoration:underline;
}
```

Step 6: Home Component

home.component.ts

```
import {
Component,
OnInit,
OnDestroy
} from '@angular/core';
@Component({
selector:'app-home',
standalone:true,
templateUrl:'./home.component.html',
styleUrls:['./home.component.css']
})
```

```
export class HomeComponent
implements OnInit,OnDestroy{
portalName="Student Course Portal";
message="Welcome to Angular 20";
searchTerm="";
isPortalActive=true;
students=350;
courses=25;
faculty=18;
ngOnInit(){
console.log(
"Home Initialized"
)
}
ngOnDestroy(){
console.log(
"Home Destroyed"
);
}
enroll(){
alert(
"Enrollment Successful"
);
}
}
```

---

home.component.html

```
<h2>
{{portalName}}
</h2>
<p>
{{message}}
</p>
<input
[(ngModel)]="searchTerm"
placeholder="Search">
<p>
Search :
{{searchTerm}}
</p>
<div>
Total Students :
{{students}}
</div>
<div>
Courses :
{{courses}}
</div>
<div>
Faculty :
{{faculty}}
</div>
<button
```

```
(click)="enroll()">
Enroll Now
</button>
```

---

home.component.css

```
h2
color:#1565c0;
}
input{
padding:8px;
width:250px;
margin:10px 0;
}
button{
padding:10px 18px;
margin-top:10px;
}
```

Step 7: Course Card Component

course-card.component.ts

```
import {
Component,
Input,
Output,
EventEmitter
} from '@angular/core';
@Component({
```

```
selector:'app-course-card',
standalone:true,
templateUrl:'./course-card.component.html',
styleUrls:['./course-card.component.css']
})
export class CourseCardComponent{
@Input()
course:any;
@Output()
enrollRequested=
new EventEmitter<number>();
enroll(){
this.enrollRequested.emit(
this.course.id
);
}
}
```

---

course-card.component.html

```
<div class="card">
<h3>
{{course.name}}
</h3>
<p>
Code :
{{course.code}}
</p>
```

```
<p>
Credits :
{{course.credits}}
</p>
<button
(click)="enroll()">
Enroll
</button>
</div>
```

---

course-card.component.css

```
.card{
border:1px solid gray;
padding:20px;
margin:15px;
border-radius:10px;
box-shadow:0 2px 5px rgba(0,0,0,.2);
}
```

Step 8: Course List Component

course-list.component.ts

```
import { Component } from '@angular/core';
@Component({
 selector: 'app-course-list',
 standalone: true,
 templateUrl: './course-list.component.html',
 styleUrls: ['./course-list.component.css']
})
```

```
export class CourseListComponent {

 courses = [
  {
    id: 1,
    name: 'Angular',
    code: 'ANG101',
    credits: 4
  },
  {
    id: 2,
    name: 'React',
    code: 'RCT201',
    credits: 3
  },
  {
    id: 3,
    name: 'Spring Boot',
    code: 'SPR301',
    credits: 5
  },
  {
    id: 4,
    name: 'Java',
    code: 'JAVA101',
    credits: 4
  },
```

```
  {
    id: 5,
    name: 'MySQL',
    code: 'SQL101',
    credits: 2
  }
 ];

 selectedCourse = 0;

 enroll(id: number) {
  this.selectedCourse = id;
 }
}
```

---

course-list.component.html

```
<h2>Available Courses</h2>

<app-course-card
 *ngFor="let c of courses"
 [course]="c"
 (enrollRequested)="enroll($event)">
</app-course-card>

<p>
 Selected Course ID :
 {{selectedCourse}}
</p>
```

Step 9: Student Profile Component

student-profile.component.ts

```
import { Component } from '@angular/core';
@Component({
 selector: 'app-student-profile',
 standalone: true,
 templateUrl: './student-profile.component.html',
 styleUrls: ['./student-profile.component.css']
})
export class StudentProfileComponent {

 student = {
  id: 101,
  name: 'John Doe',
  branch: 'Computer Science',
  semester: 5,
  cgpa: 8.75
 };

}
```

---

student-profile.component.html

```
<h2>Student Profile</h2>
<p>ID : {{student.id}}</p>
<p>Name : {{student.name}}</p>
<p>Branch : {{student.branch}}</p>
<p>Semester : {{student.semester}}</p>
```

```
<p>CGPA : {{student.cgpa}}</p>
```

---

student-profile.component.css

```
p {
 font-size: 16px;
 margin: 8px 0;
}
```

Output Screenshot: